

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни «Організація баз даних» на тему

Нормалізація бази даних

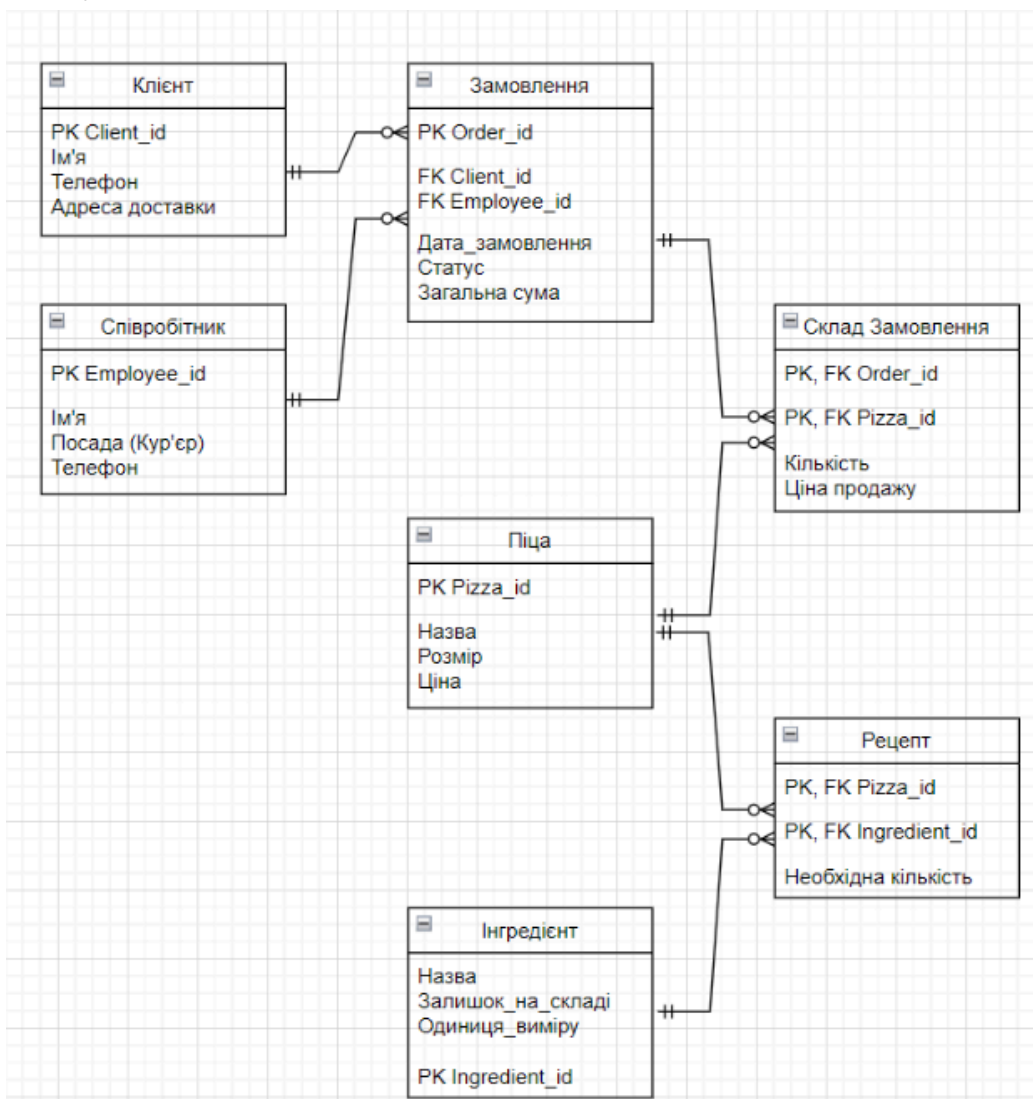
Виконала: Мишко Н. П.
Група: ІО-46
Залікова книжка: № 4608
Викладач: Русінов В. В.

Мета роботи:

- Пошук надлишковості та аномалій: виявлення потенційної надлишковості даних (наприклад, повторювані значення) або аномалій оновлення (проблеми вставки/оновлення/видалення) у поточній схемі.
- Перелік функціональних залежностей: визначте та перелічіть функціональні залежності (ФЗ) для кожної проблемної таблиці.
- Перевірка нормальних форм: оцініть поточну нормальну форму кожної таблиці (1NF, 2NF, 3NF) на основі її функціональних залежностей (ФЗ) та структури ключа.
- Застосування нормалізації: перетворення таблиць у вищі нормальні форми (до 3НФ) для усунення часткових та транзитивних залежностей.

Хід роботи

Пошук надлишковості та аномалій



На основі аналізу початкової ER-діаграми (Рисунок 1) було виявлено ряд критичних недоліків у структурі даних, які можуть призвести до аномалій:

- Надлишковість у таблиці «Клієнт»:
Поле «Адреса доставки» жорстко прив'язане до сутності клієнта. Це створює проблему, якщо один клієнт захоче зробити замовлення на різні адреси (наприклад, додому та в офіс). Доведеться або створювати дублікат клієнта з іншою адресою, або постійно переписувати існуючу (аномалія оновлення).
- Транзитивна залежність у таблиці «Співробітник»:
Поле «Посада (Кур'єр)» містить текстові значення. Якщо в майбутньому назва посади зміниться (наприклад, на «Водій-доставщик»), нам доведеться змінювати ці значення в кожному рядку таблиці співробітників. Це класична аномалія оновлення.
- Аномалія видалення у таблиці «Склад Замовлення»:
Якщо ми захочемо видалити інформацію про конкретну піцу з асортименту, ми ризикуємо втратити історичні дані про замовлення, в яких ця піца фігурувала, якщо зв'язки налаштовані некоректно.

Перелік функціональних залежностей

Для кожної проблемної таблиці визначено мінімальний набір залежностей, де ліва частина (детермінант) однозначно визначає праву частину:

Таблиця «Клієнт»:

- Client_id - Ім'я, Телефон, Адреса доставки
- Телефон - Ім'я, Адреса доставки

Таблиця «Співробітник»:

- Employee_id - Ім'я, Посада, Телефон
- Посада - Відділ

Таблиця «Склад Замовлення»:

- Order_id, Pizza_id - Кількість, Ціна продажу
- Ця таблиця має складений ключ, і ми повинні переконатися, що неключові атрибути залежать від обох частин ключа

Перевірка нормальних форм

Початкова схема бази даних (до етапу декомпозиції) знаходилася у 2NF (Другій нормальній формі).

- 1NF (Виконується): Усі значення в клітинках атомарні (типи ENUM та VARCHAR використовуються коректно, масивів або списків у комірках немає).
- 2NF (Виконується): Усі основні таблиці мають прості первинні ключі (SERIAL PRIMARY KEY). Оскільки ключ складається лише з одного стовпця, часткова залежність від ключа неможлива.
- 3NF (Порушується): Наявні транзитивні залежності. У початковій таблиці orders (Замовлення) персональні дані клієнта (ім'я, адреса) залежать від його номера телефону, а не безпосередньо від order_id. Також дані співробітника (ім'я, посада) залежать від employee_id, а не від order_id.

Застосування нормалізації

Щоб привести базу даних до 3NF, проведемо декомпозицію:

Нормалізація даних клієнтів: Виділяємо персональні дані замовників в окрему таблицю, щоб уникнути дублювання інформації (імені, адреси, телефону), якщо одна й та сама людина робить замовлення кілька разів.

- Створюємо client: (PK) client_id, first_name, last_name, phone_number, address.
- Оновлюємо orders: видаляємо текстові дані клієнта, додаємо (FK) client_id.

Нормалізація даних співробітників: Виносимо інформацію про персонал в окрему таблицю, щоб уникнути повторення їхніх імен та посад у кожному обробленому чеку.

- Створюємо employee: (PK) employee_id, first_name, last_name, position, phone_number.
- Оновлюємо orders: видаляємо текстові дані працівника, додаємо (FK) employee_id.

Перероблені SQL-інструкції CREATE TABLE

Нижче наведено команди для трансформації поточної бази даних піцерії в 3NF шляхом нормалізації таблиць employee та orders.

-- Employee

-- Створення таблиці посад

```
CREATE TABLE IF NOT EXISTS job_positions (  
    position_id SERIAL PRIMARY KEY,
```

```
position_name VARCHAR(50) NOT NULL UNIQUE,  
department VARCHAR(50) NOT NULL  
);
```

```
-- Додавання колонки для зв'язку  
ALTER TABLE employee ADD COLUMN position_id INT;
```

```
-- Перенесення унікальних комбінацій (посада + відділ)  
INSERT INTO job_positions (position_name, department)  
SELECT DISTINCT position, department FROM employee;
```

```
-- Оновлення посилання у таблиці employee  
UPDATE employee e  
SET position_id = jp.position_id  
FROM job_positions jp  
WHERE e.position = jp.position_name AND e.department = jp.department;
```

```
-- Видалення старих колонок та додавання обмежень  
ALTER TABLE employee  
DROP COLUMN position,  
DROP COLUMN department,  
ALTER COLUMN position_id SET NOT NULL,  
ADD CONSTRAINT fk_employee_position FOREIGN KEY (position_id)  
REFERENCES job_positions(position_id);
```

```
-- Orders  
-- Створення таблиці персональних даних клієнтів  
CREATE TABLE IF NOT EXISTS client (  
phone_number VARCHAR(12) PRIMARY KEY,  
first_name VARCHAR(25) NOT NULL,  
last_name VARCHAR(30) NOT NULL,  
address TEXT NOT NULL  
);
```

```
-- Перенесення даних клієнтів (унікальні записи по телефону)  
INSERT INTO client (phone_number, first_name, last_name, address)
```

```
SELECT DISTINCT client_phone, client_first_name, client_last_name,
client_address FROM orders;
```

-- Змінення структури таблиці orders

```
ALTER TABLE orders RENAME COLUMN client_phone TO phone_number;
```

```
ALTER TABLE orders
```

```
  DROP COLUMN client_first_name,
```

```
  DROP COLUMN client_last_name,
```

```
  DROP COLUMN client_address,
```

```
  ADD CONSTRAINT fk_order_client FOREIGN KEY (phone_number)
REFERENCES client(phone_number);
```

ERD після нормалізації

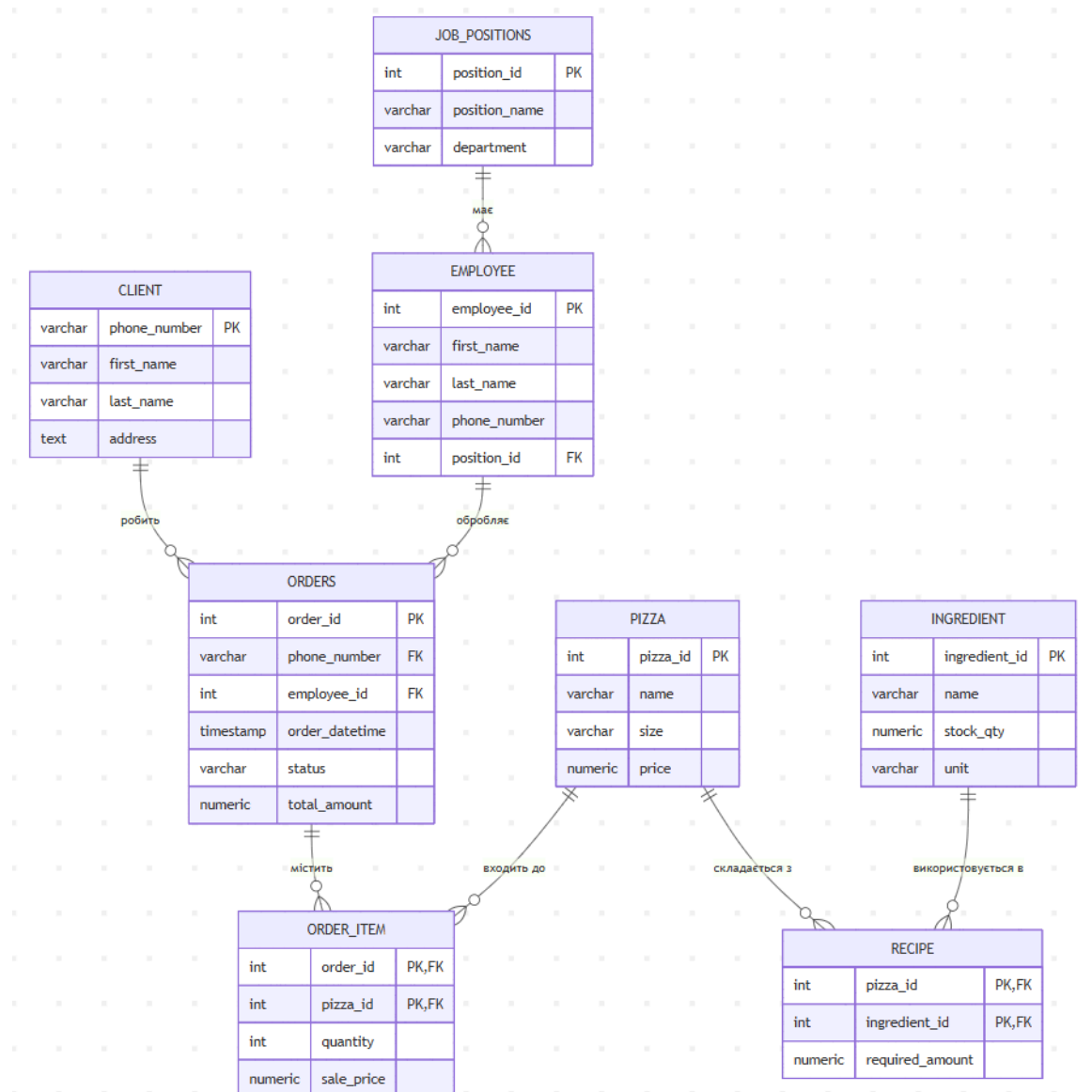


Рисунок 2 – Оновлена нормалізована схема даних піцерії

Висновки

У ході лабораторної роботи було проведено повний аналіз схеми бази даних піцерії. Шляхом виявлення транзитивних функціональних залежностей було встановлено, що початкова схема відповідала лише 2NF. Завдяки декомпозиції таблиць employee та orders на більш дрібні сутності, було досягнуто Третьої нормальної форми (3NF). Це дозволило:

- Усунути дублювання персональних даних та адрес доставки клієнтів при кожному новому замовленні.
- Спростити управління структурою персоналу (посадами та відділами) через окремий довідник.
- Мінімізувати ризик виникнення аномалій вставки, видалення та оновлення даних, що гарантує цілісність та надійність усієї бази даних.